

2.2 Typumwandlung

Yusuf hat den Code rechts entworfen. Er führt das Programm aus und erhält die Bildschirmausgaben. Zwei fehlen.

Vorhersage: Trage die fehlenden Ausgaben ein und erkläre in einem Satz, wie sie zustande kommen.

<code>println(3 + 5);</code>	Ausgabe: 8
<code>println("3" + "5");</code>	Ausgabe: <u>35</u>
<code>println("Test" + 5 + 8);</code>	Ausgabe: <u>Test58</u>
<code>println("Test" + (5 + 8));</code>	Ausgabe: Test13

Ist ein Text beteiligt, hängt + die Zeichen aneinander (von links nach rechts): "Test" + 5 ergibt "Test5", dann + 8 ergibt "Test58".

Aufgabe 1 Projekt Intro

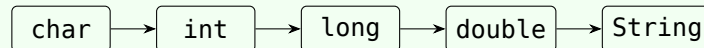
a) Ausführen und vergleichen. b) Eigene Kombinationen (TODO 1).

c) `0 + 'A'` (TODO 2): Ausgabe 65 Grund: char zählt als Zahl: 'A' hat Code 65

Implizite Typumwandlung

Bei "Test" + 5 findet eine so genannte implizite Typumwandlung statt: Die Zahl 5, üblicherweise vom Datentyp `int`, wird als String (Datentyp von Text) interpretiert. Das Plus-Zeichen ist dann kein Rechenoperator, sondern eine Verkettung (Aneinanderhängen) der Zeichen.

Die implizite Typumwandlung ist in Java nur möglich, wenn dabei keine Informationen verloren gehen. Daher gilt in Java folgende Kette:



Jeder Datentyp kann direkt in die weiter rechts stehenden umgewandelt werden.

Prüfe im Projekt: `double d = 5;` geht (5.0) `int i = 5.0;` Fehler

Anmerkung: Zu `String` wandelt Java nur beim Verketteten mit `+` um ("`" + 5`"). Eine Zuweisung wie `String s = 5;` meldet der Compiler als Fehler.

Aufgabe 2

Entferne TODO 3: `int x = 3.7;` Fehlermeldung (sinngemäß) und Grund:

Ein `double` kann nicht in eine `int`-Variable gespeichert werden. Grund: Die Nachkommastellen (.7) gingen dabei verloren.

Wie könnte man Java sagen „ich nehme den Verlust in Kauf“? Idee:

den Zieltyp ausdrücklich davorschreiben: `int x = (int) 3.7;`

Explizite Typumwandlung

Der gewünschte Zieldatentyp wird in runden Klammern vor den umzuwandelnden Wert geschrieben (englisch: *cast*):

```
int i = (int) 3.7;    char c = (char) 65;    int k = (int) 4294967298L;
```

Typumwandlung	Startwert	Endwert
<code>double → int</code>	3.0	3
<code>double → int</code>	3.7	3
<code>double → int</code>	-3.7	-3
<code>int → char</code>	65	'A'
<code>int → char</code>	66	'B'
<code>char → int</code>	'a'	97
<code>long → int</code>	300	300
<code>long → int</code>	4294967298	2 (Überlauf)
Eigene Beispiele (vorher Endwert vermuten, dann im Projekt prüfen):		
z. B. <code>double → int</code>	9.99	9
z. B. <code>int → char</code>	97	'a'

Achtung: Hierbei kann es zu Informationsverlust (nicht alles bleibt erhalten) kommen.

Nachkommastellen werden abgeschnitten, nicht gerundet.